

Dokumentation zum Projekt des Turtlebot 3

im Modul

ROBOTIK

von

Janis Schneider (780 922), Niklas von Seggern (191 623)

im Studiengang MASTER OF ENGINEERING

an der Hochschule für angewandte Wissenschaften und Kunst

Hildesheim/Holzminden/Göttingen

Fakultät Fakultät Ingenieurwissenschaften und Gesundheit Göttingen



Prüfer: Prof. Dr. rer. nat. Thomas Linkugel

Betreuer: Niklas Noack und Jan-Luca Regenhardt

24. Februar 2025

Inhaltsverzeichnis

1	Einleitung	6
1.1	Aufbau der Arbeit	6
1.2	Stand der Technik	7
1.2.1	Einsatz von ROS in der Robotik	7
1.2.2	Spurerkennung in modernen Fahrzeugen	7
2	Wissenschaftlicher Hintergrund	9
2.1	Turtlebot3	9
2.1.1	Aufbau	9
2.1.2	Sensoren	9
2.1.3	Aktoren	10
2.2	Robot Operating System (ROS)	10
2.2.1	Workspace und Catkin	11
2.2.2	Master	11
2.2.3	Nodes	12
2.2.4	Topics	12
2.2.5	Messages	12
2.2.6	Bedeutung für das Projekt	12
2.3	OpenCV	12
2.3.1	Kalibrierung der Kameraparameter	13
2.3.2	Transformationsmatrix	14
2.3.3	Techniken der Bildverarbeitung und -segmentierung	15
2.3.4	Einsatz von OpenCV im Projekt	16
3	Aufgabenstellung und Umsetzung	17
3.1	Aufgabenstellung	17
3.2	Umsetzung	18
3.2.1	Rektifizierung	18
3.2.2	Bird's-Eye-View	19
3.2.3	Bird's-Eye-View mit LiDAR-Punkten	20
3.2.4	Fahrbahn folgen und Hindernis ausweichen	22

4	Ergebnisse	26
4.1	Fazit	26
4.2	Learnings	26
4.3	Ausblick	27
5	Anhang	30

Glossar

Bird's-Eye-View ist das transformierte Originalbild in der Vogelperspektive.

Catkin Workspace ist eine Entwicklungsumgebung in ROS, in der Pakete erstellt, kompiliert und verwaltet werden. Sie besteht typischerweise aus den Verzeichnissen `src`, `devel`, `build` und `install`.

Embedded Systems sind speziell entwickelte Computersysteme, die in größere Geräte oder Maschinen integriert sind, um spezifische Funktionen zu erfüllen.

Git-Repository ist ein Versionskontrollsystem-Ordner, der Quellcode, Änderungen und Historie eines Projekts speichert. Ermöglicht verteilte Softwareentwicklung mit Branching und Merging.

Homographiematrix beschreibt die projektive Transformation zwischen zwei Ebenen.

HSV-Farbraum ist ein Farbmodell, das Farben in drei Dimensionen darstellt: Farbton (Hue), Sättigung (Saturation) und Wert (Value).

Kameramatrix ist eine 3×3 -Matrix, die die intrinsischen Parameter einer Kamera beschreibt, einschließlich Brennweite, Bildmittelpunkt und Linsenverzerrung.

LiDAR (Light Detection and Ranging) ist eine Sensortechnologie, die mittels Laserimpulsen die Entfernung zu Objekten misst, um präzise 3D-Karten der Umgebung zu erstellen.

Morphologische-Operation in OpenCV ist eine bildverarbeitende Technik, die mit Strukturelementen arbeitet und durch Erosion, Dilatation, Öffnen oder Schließen dazu dient, Rauschen zu entfernen, Kanten zu glätten oder Strukturen im Bild hervorzuheben.

Node ist die kleinste Einheit eines Prozesses, der in ROS läuft.

OpenCR (Open-source Control Module for ROS) ist ein Embedded-Board, das ROS unterstützt und als Hauptcontroller des Turtlebots dient.

PD-Regler (Proportional-Derivative-Regler) ist ein Regelungssystem, das die Regelabweichung (Fehler) proportional und deren Änderungsrate (Ableitung) berücksichtigt, um eine schnelle und stabile Reaktion ohne bleibenden Fehler zu ermöglichen.

Publisher veröffentlicht Nachrichten auf einem ROS-Topic.

Rektifikation ist ein Verfahren in der Bildverarbeitung, das Verzerrungen in Kamerabildern korrigiert, insbesondere bei Stereobildern zur besseren Disparitätsberechnung.

ROS-Client ist ein ROS-Knoten, der mit dem ROS Master kommuniziert, um Nachrichten zu senden oder zu empfangen. Clients können z. B. Sensordaten verarbeiten oder Steuerbefehle senden.

ROS-Master ist ein zentraler Knoten in ROS, der die Kommunikation zwischen anderen ROS-Knoten koordiniert, indem er Namensauflösungen und Registrierungen verwaltet.

ROS-Topic ist ein Kommunikationskanal in ROS, über den Nachrichten zwischen Knoten asynchron ausgetauscht werden. Ein Topic kann von mehreren Publishern und Subscribern genutzt werden.

State-Machine (Zustandsmaschine) ist ein abstraktes Modell zur Steuerung von Abläufen, bei dem ein System zu jedem Zeitpunkt in einem definierten Zustand ist und durch Eingaben oder Ereignisse zwischen Zuständen übergangsgesteuert wechselt.

Subscriber abonniert Nachrichten von einem ROS-Topic.

TOF (Time of Flight) ist ein Messprinzip, bei dem die Zeit gemessen wird, die ein Lichtimpuls benötigt, um von einem Objekt reflektiert und zurück zum Sensor zu gelangen. Die gemessene Zeit wird verwendet, um die Distanz zum Objekt zu berechnen.

Turtlebot ist ein kleiner Roboter der mit ROS programmiert wird.

Two-Line-Following ist ein Verfahren, bei dem ein Roboter zwei Linien auf dem Boden verfolgt, um sich autonom entlang einer definierten Strecke zu bewegen.

Verzerrungskoeffizienten beschreiben, wie stark und in welcher Form das Bild verzerrt ist. Die Verzerrung des Bildes wird durch radiale und tangentialen Verzerrungskoeffizienten beschrieben.

Abkürzungsverzeichnis

IMU Inertial Measurement Unit.

Odom Odometry.

OpenCV Open Source Computer Vision Library.

ROS Robot Operating System.

SLAM Simultaneous Localization and Mapping.

TOF Time of Flight.

1 Einleitung

Autonome Systeme bzw. Roboter spielen heutzutage eine immer größer werdende Rolle. Sowohl in der Industrie, etwa in Fertigungsstraßen, als auch zunehmend in unserem Alltag, wie beim autonomen Fahren von Autos oder dem Staubsaugroboter werden sie immer häufiger eingesetzt. Besonders im Bereich des automatisierten Fahrens stellt die Robotik einen fundamentalen Baustein dar. Dabei ist das Zusammenspiel von Sensoren und Aktoren von zentraler Bedeutung. Die Sensoren messen physikalische Größen und wandeln diese in digitale oder elektrische Signale um, während die Aktoren Steuerbefehle wie Bewegungsgeschwindigkeiten oder Richtungen in physische Bewegungen umsetzen. Um entsprechende autonome Systeme entwickeln zu können ist es daher entscheidend, dass Roboter in der Lage sind, ihre Umwelt wahrzunehmen, zu verstehen und intelligent darauf zu reagieren [1, S. 399]. Diese Fähigkeiten sind essentiell für eine Vielzahl von Anwendungen in der Robotik, von der autonomen Navigation in Lagerhallen bis hin zur Unterstützung von Menschen in Serviceumgebungen.

Im Rahmen des Moduls Robotik haben wir daher ein entsprechendes Projekt bearbeitet:

Ziel des Projektes war die Entwicklung eines autonomen Robotersystems auf Basis des Turtlebot3 Burger, welcher sich in einer vorgegebenen Umgebung ein Two-Line-Following durchführen, Objekte erkennen und Hindernissen ausweichen soll.

Da das Projekt in erster Linie dazu dient, den Umgang mit dem Robot Operating System (ROS) zu erlernen, bietet der Turtlebot eine passende Plattform für das Projekt. Er ist klein, kostengünstig und für den Einsatz in Bildung und Forschung ausgelegt [2].

Ein Kernpunkt des Two-Line-Following war die Erkennung der Linien. Dazu wurden die Kalibrierung, Transformation und Auswertung der Kameradaten mit Open Source Computer Vision Library (OpenCV) durchgeführt, da OpenCV eine einfache kostenlose Nutzung von zahlreichen Algorithmen im Bereich der Bildverarbeitung und Computer-Vision ermöglicht [3]. Um die relevanten Informationen filtern und weiterverarbeiten zu können, war vor allem die Transformation in die Bird's-Eye-View entscheidend. Zur Hinderniserkennung und -vermeidung kam zudem ein LiDAR-Sensor des Turtlebot zum Einsatz.

1.1 Aufbau der Arbeit

Im Rahmen dieser Arbeit geben wir zunächst einen Überblick über den aktuellen Stand der Technik (Unterabschnitt 1.2). Anschließend erläutern wir den wissenschaftlichen Hintergrund zu den

verwendeten Systemen, Begriffen, Bibliotheken und Algorithmen (Abschnitt 2), bevor wir zur konkreten Aufgabenstellung und Umsetzung im Projekt übergehen (Abschnitt 3). Zuletzt stellen wir unsere Ergebnisse vor und bieten einen Ausblick zur Fortführung des Projekts (Abschnitt 4).

1.2 Stand der Technik

1.2.1 Einsatz von ROS in der Robotik

Das ROS hat sich als Standard in der Robotik etabliert und ermöglicht die Entwicklung komplexer Roboteranwendungen durch seine flexible, offene und umfangreiche Architektur. Es bietet eine Vielzahl von Software-Bibliotheken und Tools, die die Integration von Sensoren und Aktoren erleichtern und somit die Entwicklung autonomer Systeme beschleunigen. [2]

Auch in der Automobilindustrie findet ROS Anwendung. So nutzt Porsche Engineering ROS in ihren JUPITER-Versuchsfahrzeugen, um neue automatisierte Fahrfunktionen schneller zu entwickeln und zu testen. Durch die Integration von ROS können Sensoren wie LiDAR und Kameras effizient eingebunden und neue Softwarefunktionen flexibel implementiert werden. Dies verkürzt die Entwicklungszeiten und ermöglicht eine schnelle Prototypenerstellung für innovative Fahrassistenzsysteme [4].

1.2.2 Spurerkennung in modernen Fahrzeugen

Die Spurerkennung ist eine zentrale Funktion moderner Fahrassistenzsysteme, insbesondere für Spurhalteassistenten. Sie ermöglicht es dem Fahrzeug, die Fahrspur zu identifizieren und innerhalb dieser zu bleiben, was die Sicherheit und den Komfort für den Fahrer erhöht. [5]

Kamerabasierte Spurerkennung

Die gängigste Methode zur Spurerkennung basiert auf der Verwendung von Digitalkameras, die in der Fahrzeugfrontscheibe integriert sind. Diese Kameras erfassen kontinuierlich Bilder der Fahrbahn, die anschließend mittels Mustererkennung analysiert werden, um Fahrbahnmarkierungen und andere relevante Merkmale zu identifizieren. Fortgeschrittene Systeme nutzen dabei mehrere Kameras mit unterschiedlichen Blickwinkeln, um eine dreidimensionale Erfassung der Fahrspur zu ermöglichen. Die erkannten Fahrspurdaten werden mit Fahrzeugsensordaten wie Geschwindigkeit und Lenkwinkel kombiniert, um den aktuellen und zukünftigen Verlauf der Fahrspur zu bestimmen und gegebenenfalls korrigierende Lenkbewegungen einzuleiten. [6]

Alternative Sensortechnologien

Neben der kamerabasierten Erkennung existieren weitere Sensortechnologien zur Spurerkennung:

- **Infrarot-Linienerkennung:** Sensoren am Unterboden des Fahrzeugs erkennen das Überfahren von Fahrbahnmarkierungen. Diese Technologie eignet sich hauptsächlich für Spurverlassenswarnungen und weniger für aktive Spurhaltesysteme, da sie den Fahrspurverlauf nicht vorausschauend erfassen kann. [7]
- **Lidar-Systeme:** Obwohl in Serienfahrzeugen noch nicht weit verbreitet, bieten LiDAR-Systeme das Potenzial, durch die Erfassung der Umgebung in drei Dimensionen eine präzise Spurerkennung zu ermöglichen. [7]

Die Integration dieser Sensordaten mit fortschrittlichen Bildverarbeitungsalgorithmen ermöglicht es modernen Fahrzeugen, die Fahrspur zuverlässig zu erkennen und entsprechend zu reagieren.

2 Wissenschaftlicher Hintergrund

2.1 Turtlebot3

2.1.1 Aufbau

Der Turtlebot Burger ist eine kompakte, kostengünstige und anpassbare mobile Roboterplattform, die speziell für den Einsatz in der Forschung, Bildung und Produktentwicklung konzipiert wurde. Er basiert auf ROS und bietet eine vielseitige Entwicklungsumgebung für Robotikprojekte. [2]

Der Turtlebot wurde als modulare Plattform entwickelt, die einfach erweitert und mit zusätzlichen Sensoren oder Recheneinheiten aufgerüstet werden kann. Dank seiner geringen Größe und seines leichten Gewichts eignet er sich ideal für den Einsatz in Innenräumen und für autonome Navigationsaufgaben. [2]

Die Firmware läuft auf dem OpenCR (Open-source Control Module for ROS)-Board. Sie ist das Herzstück der Motorsteuerung und verwaltet die Kommunikation mit den Sensoren und Aktoren des Roboters. Die Firmware schätzt die Position des Roboters anhand der Encoder-Werte der Dynamixel-Motoren und steuert die Geschwindigkeit basierend auf den Befehlen der übergeordneten Software. Die Firmware enthält auch einen OpenCV-Controller zur Verarbeitung von Kamerabildern. [1, S. 283]

2.1.2 Sensoren

Der Turtlebot ist mit verschiedenen Sensoren ausgestattet, die es ihm ermöglichen, seine Umgebung wahrzunehmen und sich darin zu orientieren. Zu den wichtigsten Sensoren gehören:

- **Kamera:** Der Turtlebot Burger verfügt standardmäßig über keine Kamera. Er kann jedoch mit einer Raspberry Pi Camera Module v2.1 erweitert werden. [2] Die RGB-Kamera dient der visuellen Erfassung der Umgebung. Sie kann für Aufgaben wie Objekterkennung, Linienverfolgung und Hinderniserkennung eingesetzt werden.
- **LiDAR:** Der Turtlebot verwendet den 360° Laser Distance Sensor (LDS-02), um seine Umgebung zu scannen und eine 2D-Kartierung für Simultaneous Localization and Mapping (SLAM) zu ermöglichen. [1, S. 293] Die Entfernungsdaten können ebenfalls für die Lokalisierung und Hindernisvermeidung verwendet werden. Der LiDAR-Sensor misst Distanzen mithilfe des Time of Flight (TOF) Prinzips, bei dem ein Lichtimpuls ausgesendet und der reflektierte Impuls detektiert wird. Die gemessene Zeit zwischen Senden und Empfangen

des Pulses ermöglicht die Berechnung der Distanz. Der Turtlebot nutzt einen zweidimensionalen LiDAR-Sensor, der sich um 360° dreht und Messpunkte mit einer Auflösung von 1° aufnimmt. Die maximale Messdistanz beträgt 3,5 m. [2]

- **Odom:** Die Position des Roboters wird durch eine Kombination aus 3-Achsen-Gyroskop und 3-Achsen-Beschleunigungssensor erfasst. Die Odometry (Odom) liefert Informationen über die Bewegung des Roboters, basierend auf den Daten der Encoder und der Inertial Measurement Unit (IMU). Die Odom ist essenziell für die Navigation, da sie Informationen über die Position und Orientierung des Roboters liefert. [1, 293 f.] Die Positionsbestimmung ist allerdings nicht zwangsläufig exakt, sondern wird aus den Sensordaten geschätzt.

2.1.3 Aktoren

Die Bewegungssteuerung des Turtlebot Burger erfolgt über *DYNAMIXEL XL430-W250* Servomotoren. [2] Durch Ansteuerung der Motoren kann der Roboter sich vorwärts und rückwärts bewegen sowie drehen. Die Ansteuerung der Motoren erfolgt über die `cmd_vel`-Topic, wobei die translatorische und rotatorische Geschwindigkeit in Meter pro Sekunde bzw. Radiant pro Sekunde angegeben wird. Für eine sichere Bewegung ist eine präzise Steuerung der Aktoren erforderlich. [1, S. 292]

2.2 Robot Operating System (ROS)

ROS ist ein Open-Source-Framework für Robotik-Anwendungen. Es stellt eine Sammlung von Softwarebibliotheken und Werkzeugen bereit, die zum Aufbau von Roboteranwendungen dienen. ROS ist kein Betriebssystem im herkömmlichen Sinne, sondern ein Meta-Betriebssystem, das auf einem bestehenden Betriebssystem wie Linux (Ubuntu) aufbaut. Es bietet Dienste wie Hardware-Abstraktion, Geräte-Treiber, Implementierung häufig verwendeter Funktionen, Nachrichtenübermittlung zwischen Prozessen und Paketverwaltung an. [1, S. 7–11]

Das ROS-Dateisystem ist in verschiedene Ebenen unterteilt, die die Organisation von Software und Prozessen betreffen. Nachfolgend sind die wichtigsten Komponenten des Dateisystems zusammengefasst [8]:

- **Pakete:** ROS organisiert Software in Paketen, die die kleinste Einheit für den Aufbau und die Veröffentlichung von Software darstellen. Ein Paket kann verschiedene ROS-Runtime-Prozesse (Nodes), Bibliotheken, Konfigurationsdateien und mehr enthalten. Pakete sind die grundlegenden Bausteine für die Entwicklung und Freigabe von ROS-Software.

- **Metapackages:** Diese speziellen Pakete dienen dazu, mehrere verwandte Pakete zu gruppieren. Sie sind häufig als Platzhalter für ROS-Bausteine von älteren Versionen gedacht.
- **Paket-Manifeste:** Die Datei `package.xml` enthält Metadaten über ein Paket, wie Name, Version, Beschreibung, Lizenz und Abhängigkeiten.
- **Repositories:** Ein Repository ist eine Sammlung von Paketen, die ein gemeinsames Versionskontrollsystem (VCS) teilen und oft zusammen veröffentlicht werden.
- **Message- und Service-Dateien:** ROS verwendet Nachrichten (`msg`) und Dienste (`srv`), um Datenstrukturen für die Kommunikation zwischen Nodes zu definieren. Nachrichten enthalten die Felder, die in der Kommunikation zwischen Nodes verwendet werden.

2.2.1 Workspace und Catkin

Der ROS-Workspace dient nicht nur zur Speicherung benutzerdefinierter Pakete, sondern wird auch mit einer Standardstruktur aus den Ordnern `src`, `build` und `devel` organisiert. Der `src`-Ordner enthält den Quellcode der Pakete, der `build`-Ordner speichert die Build-Dateien, und der `devel`-Ordner behält die entwickelten Bibliotheken sowie ausführbare Dateien. Für den Build-Prozess verwendet ROS das Catkin-Build-System, das auf CMake basiert. Die Konfiguration des Builds erfolgt in der Datei `CMakeLists.txt` innerhalb des jeweiligen Paket-Ordners. [1, S. 49]

Die ROS-Architektur basiert auf den Konzepten Master, Nodes, Topics, Messages und Services. Ein grundlegender Aufbau ist in Abbildung 5 im Anhang zu sehen. Diese wird im folgenden weiter erläutert.

2.2.2 Master

Der Master (`roscore`) ist das Herzstück des ROS-Netzwerks. Er fungiert als Server für Node-to-Node-Verbindungen und Message Communication. Nodes melden sich beim Master an und erhalten Informationen zur Kommunikation untereinander (TCP/IP) (siehe Abbildung 5). Ohne den Master ist keine Verbindung zwischen Nodes und keine Message Communication möglich. Beim Ausführen von ROS wird der Master mit der URI-Adresse und dem Port konfiguriert. Für unser Projekt haben wir den Raspberri Pi auf dem Turtlebot als Master festgelegt. [1, S. 41]

2.2.3 Nodes

Ein Node ist die kleinste Einheit eines Prozesses, der in ROS läuft. Er kann als ein eigenständiges Programm betrachtet werden. ROS empfiehlt, für jede Aufgabe einen einzelnen Node zu erstellen, um die Wiederverwendbarkeit zu gewährleisten. Nodes können Publisher (pub), Subscriber (sub) oder beides sein, wie in Abbildung 5 zu sehen ist. [1, S. 42] ROS ermöglicht es zudem, Nodes in verschiedenen Sprachen zu schreiben und Informationen über Message Communication auszutauschen. Dies fördert die Wiederverwendbarkeit von Code und die Flexibilität bei der Entwicklung. ROS unterstützt verschiedene Programmiersprachen wie Python, C++, Lisp, Java, Lua, .NET, EusLisp und R. [1, S. 68]

2.2.4 Topics

Topics sind Themenbezeichnungen, unter denen Nachrichten (Messages) ausgetauscht werden. Nodes abonnieren (subscribe) oder veröffentlichen (publish) Nachrichten unter einem Topic. Topics ermöglichen eine unidirektionale, asynchrone Kommunikation. Ein Node kann mehrere Topics abonnieren, und ein Topic kann von mehreren Nodes abonniert werden (siehe Abbildung 5). [1, S. 43] Die Nodes müssen sich dabei untereinander nicht kennen.

2.2.5 Messages

Messages sind Nachrichten, die von Nodes über ein Topic ausgetauscht werden. Sie bestehen aus Datentypen und Bezeichnern. Messages können einfache Datentypen, Arrays oder komplexe Datenstrukturen enthalten. [1, S. 42–43]

2.2.6 Bedeutung für das Projekt

ROS bietet eine Infrastruktur und Werkzeuge für die Entwicklung von Roboteranwendungen. Es vereinfacht die Integration von Hardware und Software und fördert die Wiederverwendbarkeit von Code. Durch die Kommunikation über Topics können erstellte Nodes problemlos für die folgenden Aufgaben verwendet werden. Nach einer gewissen Einarbeitungszeit, kann daher effizienter gearbeitet werden.

2.3 OpenCV

OpenCV ist eine leistungsfähige Open-Source-Computer-Vision-Bibliothek, die in C und C++ geschrieben ist und unter Linux, Windows und Mac OS X läuft. Sie ist für hohe Recheneffizienz und

Echtzeitanwendungen optimiert und kann Multicore-Prozessoren nutzen. OpenCV enthält über 500 Funktionen für Bildverarbeitung, Kamera-Kalibrierung, Robotik und weitere Aufgaben in der Bild- und Videoanalyse. [3, S. 2]

Computer Vision bezeichnet dabei die Umwandlung von Bild- oder Videodaten in Entscheidungen oder neue Darstellungen, um bestimmte Ziele zu erreichen. Während das menschliche Gehirn visuelle Signale durch komplexe Mechanismen verarbeitet und mit Erfahrungen verknüpft, erhält ein Computer lediglich eine Matrix von Zahlen, ohne eingebautes Mustererkennungsvermögen oder Vorwissen. Die Herausforderung besteht nun darin, diese Rohdaten in sinnvolle Informationen zu übersetzen, etwa die Identifikation eines Objekts. [3, S. 2–4]

Bei der klassischen Aufnahme mit einer Lochkamera entstehen keine Verzerrungen. Allerdings werden in der heutigen Praxis Linsen verwendet, um mehr Licht und ggf. einen weiteren Bildbereich einzufangen. Dadurch entstehen jedoch Bildverzerrungen. Diese müssen mit der Kamerakalibrierung korrigiert werden, um präzisere und realistischere Bildanalysen zu ermöglichen. [3, S. 370–371]

2.3.1 Kalibrierung der Kameraparameter

Intrinsische Kameraparameter:

Die intrinsischen Parameter einer Kamera beschreiben deren interne Eigenschaften, wie die Geometrie (Brennweite, Brennmittelpunkt) und Linsenverzerrungen. Diese Parameter sind in der Kameramatrix K zusammengefasst, die wie folgt aussieht [3, S. 374]:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Hierbei sind f_x und f_y die Brennweiten in Pixeln entlang der x- und y-Achse, und c_x und c_y die Koordinaten des Hauptpunkts (normalerweise der Bildmittelpunkt). [3, 373 f.]

Extrinsische Kameraparameter:

Die extrinsischen Parameter definieren die Position und Orientierung der Kamera in den Weltkoordinaten. Sie bestehen aus einer Rotationsmatrix R und einem Translationsvektor t , die zusammen die Transformation von Weltkoordinaten zu Kamerakoordinaten beschreiben:

$$[R \mid t] \quad (2)$$

Die vollständige Projektionsmatrix P kombiniert die intrinsischen und extrinsischen Parameter [3,

S. 379]:

$$P = K \cdot [R \mid t] \quad (3)$$

Rektifizierung:

Die Rektifizierung dient dazu, Verzerrungen zu korrigieren, die durch das Kameraobjektiv (also die intrinsischen Kameraparameter) verursacht werden. Diese Verzerrungen werden durch radiale und tangentielle Verzerrungskoeffizienten modelliert. Aufgrund seiner klar definierten Ecken und Linien eignet sich ein Schachbrettmuster besonders gut zur Bestimmung der Koeffizienten für die Rektifizierung. Für die Rektifizierung wird wie folgt vorgegangen [3, 374 f.]:

1. **Bestimmung der Verzerrungskoeffizienten:** Diese Koeffizienten beschreiben, wie stark und in welcher Form das Bild verzerrt ist (Bestimmung der Koeffizienten z.B. mithilfe des Schachbrettmusters). Die Verzerrung des Bildes wird durch radiale (k_1, k_2, k_3) und tangentielle (p_1, p_2) Verzerrungskoeffizienten beschrieben [3, S. 376]:

$$x_{corr} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) = x + (2p_1 y + p_2(r^2 + 2x^2)) \quad (4)$$

$$y_{corr} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) = y + (2p_1 x p_2(r^2 + 2y^2)) \quad (5)$$

Durch Anwendung dieser Gleichungen kann die Verzerrung berechnet und korrigiert werden.

2. **Erstellung von Rektifizierungsmatrizen:** Anhand der Verzerrungskoeffizienten und der Kameramatrix werden Mapping-Tabellen erstellt, die für jedes Pixel im verzerrten Bild die Position im entzerrten Bild angeben.
3. **Anwendung der Rektifizierung:** Das ursprüngliche Bild wird mithilfe der Mapping-Tabellen transformiert, um ein entzerrtes Bild zu erhalten.

2.3.2 Transformationsmatrix

Um die Perspektive eines Bildes in eine Bird's-Eye-View zu transformieren, wird eine Homographie-Matrix H verwendet. Diese Matrix beschreibt die projektive Transformation zwischen zwei Ebenen. Dazu sind folgende Schritte notwendig:

1. **Auswahl im Originalbild:** Hier werden vier Punkte ausgewählt, die ein Viereck definieren, das den Bereich repräsentiert, der in die Vogelperspektive transformiert werden soll.

2. **Definition in der Zielansicht:** Diese Punkte definieren, wie das Viereck in der Vogelperspektive aussehen soll (z.B. ein Rechteck mit bekannten Abmessungen).
3. **Berechnung der Homographie-Matrix:** Anhand der Korrespondenz zwischen den Punkten im Originalbild und den Punkten in der Zielansicht wird die Homographie-Matrix berechnet.

Transformation: Nachdem die Homographie-Matrix H berechnet wurde, kann sie verwendet werden, um das gesamte Bild in die Bird's-Eye-View zu transformieren.

Die Transformation wird auf jeden Pixel des Originalbildes angewendet, um dessen neue Position im transformierten Bild zu bestimmen. Dies erfolgt durch die Matrixmultiplikation [9]:

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = H \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (6)$$

Nach der Transformation werden die neuen Koordinaten durch die Normalisierung mit w berechnet:

$$x' = \frac{x'}{w}, \quad y' = \frac{y'}{w} \quad (7)$$

Das resultierende Bild wird anschließend interpoliert, um eine gleichmäßige Darstellung der transformierten Pixelwerte zu gewährleisten. [9]

2.3.3 Techniken der Bildverarbeitung und -segmentierung

OTSU-Methode: Die OTSU-Methode wählt einen Schwellenwert, der die Intraklassenvarianz der beiden resultierenden Pixelgruppen (Hintergrund und Vordergrund) minimiert. [10] In OpenCV erfolgt die Anwendung der OTSU-Methode mit der Funktion `cv2.threshold()`.

HSV-Farbraum: Der HSV-Farbraum (Hue, Saturation, Value) ist ein Modell zur Darstellung von Farben, das sich von dem herkömmlichen RGB-Modell (Rot, Grün, Blau) unterscheidet. Im HSV-Modell werden Farben durch drei Parameter beschrieben: Farbton (Hue), Sättigung (Saturation) und Helligkeit (Value). Der Farbton (Hue) modelliert den eigentlichen Farbtyp, wie z.B. Rot, Blau oder Grün. Die Sättigung (Saturation) beschreibt, wie stark die Farbe gesättigt ist. Er reicht von ungesättigt (Grautönen) bis hin zu vollständig gesättigt (keine Weißkomponente). Der Wert (Value) beschreibt die Helligkeit oder Intensität der Farbe. Dieses Modell ist besonders nützlich bei Bildverarbeitungsaufgaben, bei denen Objekte anhand ihrer Farbe segmentiert werden müssen, da der Farbtonkanal eine einfache Möglichkeit bietet, Farben zu modellieren und zu vergleichen. [11] Mit der `inRange` -Funktion von OpenCV, werden alle Pixel innerhalb der über den HSV-Farbraum

vorgegebenen Maske auf 1 (weiß) und alle anderen Pixel auf 0 (schwarz) gesetzt.

Morphologische Operationen: Morphologisches Öffnen und Schließen sind Bildverarbeitungsoperationen, die auf binären Bildern basieren. Sie verwenden ein strukturierendes Element (Kernel), das Form und Größe der Transformation bestimmt. Öffnen (Erosion gefolgt von Dilatation) entfernt kleine Störungen und trennt verbundene Objekte, wodurch Rauschen reduziert wird. Schließen (Dilatation gefolgt von Erosion) füllt kleine Löcher und verbindet unterbrochene Strukturen, um Objekte zu vervollständigen und Kanten zu glätten. Während Öffnen zur Rauschunterdrückung dient, wird Schließen genutzt, um Lücken zu schließen und Formen zu verbessern. [12]

Konturen: Konturen sind Grenzen zwischen einem Objekt und dem Hintergrund in einem Bild. Sie sind die Kurven, die alle kontinuierlichen Punkte mit der selben Farbe oder Intensität verbinden. Daher lassen sie sich besonders gut in Binärbildern finden. Die OpenCV-Funktion `findContours()` kombiniert verschiedene Techniken und Algorithmen, um die Konturen der Elemente zu bestimmen. Für die Kantenerkennung in Binärbildern eignen sich ein topologischer Ansatz, der Konturen anhand der Verbindungen von Pixeln erkennt (Suzuki-Algorithmus), sowie ein Schwellenwertverfahren. [3, 234 f.]

2.3.4 Einsatz von OpenCV im Projekt

Im Rahmen dieses Projekts spielt OpenCV eine zentrale Rolle bei der Durchführung der Bildverarbeitungsschritte, die für die präzise Linienverfolgung mit dem Turtlebot erforderlich sind. Die Kamerabilder werden zunächst durch die Rektifizierung von Verzerrungen befreit, um eine realistischere Darstellung der Umgebung zu erhalten. Anschließend wird eine Bird's-Eye-View-Transformation angewendet, um die Perspektive zu ändern und eine top-down Sicht zu ermöglichen, die für die Linienverfolgung besser geeignet ist.

Die Segmentierung der Bildinhalte erfolgt mithilfe des HSV-Farbraums, um relevante Objekte aufgrund ihrer Farbe zu extrahieren. Morphologische Operationen, wie Öffnen und Schließen, werden genutzt, um Rauschen zu entfernen und die Objekte zu verfeinern, bevor die Konturen im Bild zur Erkennung und Verfolgung der Linien verwendet werden. Diese verschiedenen Funktionen von OpenCV ermöglichen eine präzise und effektive Umsetzung der Linienverfolgung, indem sie die Bildqualität verbessern und relevante Merkmale extrahieren.

3 Aufgabenstellung und Umsetzung

Wie schon in der Einleitung beschrieben, soll bei der Projektarbeit der Turtlebot so programmiert werden, dass er mithilfe der Kamera einem Kurs zwischen zwei Linien folgt. Dabei gibt es mehrere Aufgaben die umzusetzen sind. Diese sind in der Aufgabenstellung dargestellt. Im Anschluss wird im Kapitel Umsetzung die Implementierung behandelt.

3.1 Aufgabenstellung

Die Aufgabenstellung besteht aus Pflichtaufgaben und Zusatzaufgaben.

In den Pflichtaufgaben sind die nachfolgende Aufgaben enthalten.

Zu aller erst müssen die intrinsischen und extrinsischen Parameter der RGB-Kamera bestimmt werden, damit Verzerrungen des Bildes vermieden werden.

Als zweites wird mithilfe der Parameter ein rektifiziertes Bild erstellt, welches verzerrungsfrei ist. Dieses wird dann auf einem ROS-Topic publiziert.

Anschließend wird die Homographiematrix für die perspektivische Transformation vom Standardbild in die Bird's-Eye-View berechnet.

Darauffolgend wird mithilfe der Homographiematrix das Bild in der Bird's-Eye-View erstellt und auf einem ROS-Topic ausgegeben.

Die nächste Aufgabe beinhaltet das Auslesen der Punktwolke vom LiDAR-Scanner und das Bestimmen der Transformationsmatrix zwischen der Punktwolke und dem Bird's-Eye-View.

Nach dem Bestimmen der Transformationsmatrix soll nun das Bird's-Eye-View mit den eingezeichneten LiDAR-Punkten auf einem ROS-Topic publiziert werden.

Die letzte Pflichtaufgabe ist das Entwickeln eines Algorithmus der das Erkennen und Folgen einer Fahrbahn ermöglicht, welche von zwei Streifen begrenzt ist. Der Turtlebot soll zwischen den Linien fahren und wenn eine Linie unterbrochen ist trotzdem noch der Fahrbahn folgen. (vgl. [13])

Zu den Pflichtaufgaben kommen drei Zusatzaufgaben. Diese sind nachfolgend beschrieben.

Die erste Zusatzaufgabe ist das Erkennen und Umfahren von Hindernissen.

Die zweite ist das Darstellen der Trajektorie des Roboters und von Hindernissen in der Umgebung in das Visualisierungsprogramm RViz.

Die letzte Zusatzaufgabe besteht darin, eine Karte während der Fahrt zu erstellen, indem das Stitching-Verfahren auf die aufgenommenen Bilddaten angewendet wird. (vgl. [13])

3.2 Umsetzung

Zu Beginn des Projekts haben wir anhand der Aufgabenstellung einen Zeitplan (Abbildung 1) erstellt, in dem alle zu erledigenden Aufgaben festgehalten und terminiert wurden. Zudem wurden zeitliche Einschränkungen und Zusatzaufgaben entsprechend berücksichtigt.

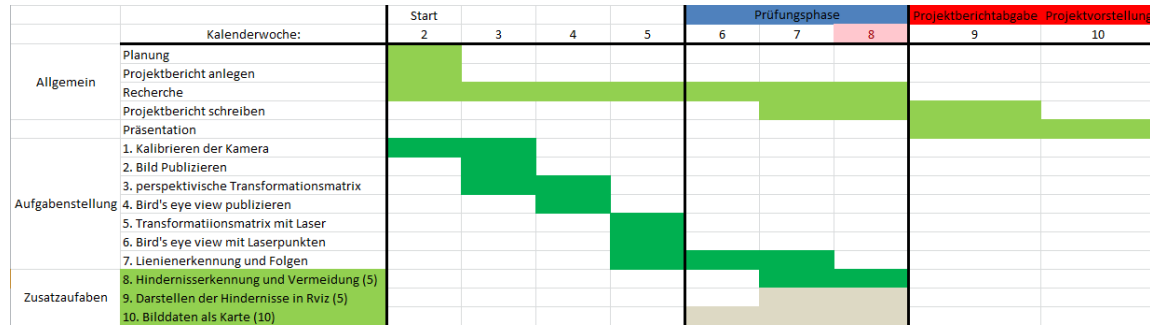


Abbildung 1: Zeitplan

Für eine geordnete Strukturierung des Projekts wurde außerdem ein Git-Repository angelegt, sodass Zwischenstände später verglichen werden konnten und eine effiziente Zusammenarbeit ermöglicht wurde.

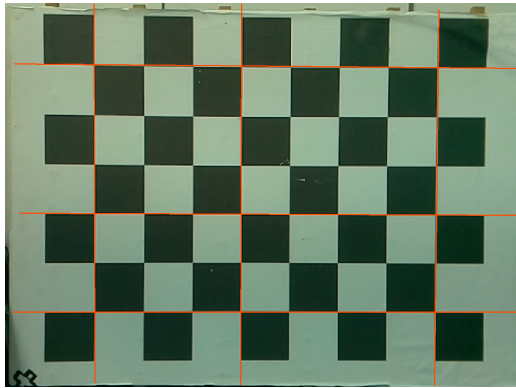
Bei der Umsetzung wurde sich an die Reihenfolge der Aufgaben gehalten. Teilweise wurden dabei mehrere Aufgaben in einem Schritt erledigt. Jede der nachfolgenden Unterkapitel entsprechen einem eigenen ROS-Node.

3.2.1 Rektifizierung

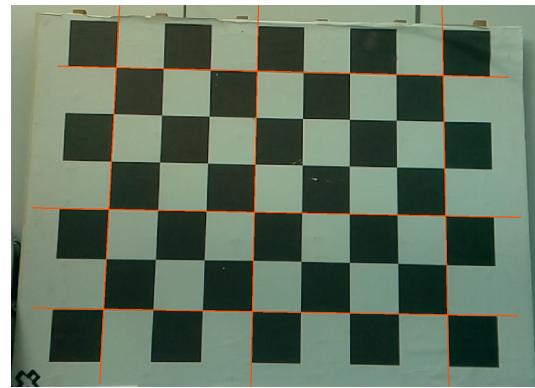
Um die intrinsischen und extrinsischen Kameraparameter zu bestimmen, wurde das Skript `cameracalibrator.py` verwendet. Dieses Tool ermöglicht die einfache Kalibrierung der Kamera, in dem ein Schachbrett vor die Kamera des Turtlebots gehalten und langsam in alle Richtungen bewegt und gekippt wird. Dabei zeigt das Tool den Fortschritt der jeweiligen Bewegungsrichtung an. Sobald alle Werte im grünen Bereich sind, kann die Kalibrierung gespeichert werden. Dabei werden zwei Dateien erstellt. Die `ost.txt` und die `ost.yaml`. Hierbei ist die `ost.yaml` besonders wichtig, da diese nun dem Turtlebot für die Kamerakalibrierung übergeben werden muss. In der Datei befinden sich neben anderen Parametern die Kameramatrix und die Verzerrungskoeffizienten. Mit dieser Datei kalibriert der Roboter die Kamera bei jedem Start der Kamera neu.

In Abbildung 2 ist einmal das Bild vor und einmal nach der Rektifizierung zu sehen.

Wie Anhand der roten Linien visualisiert wurde, ist kaum ein Unterschied zwischen dem Bild vor



(a) Schachbrettmuster vor der Kalibrierung



(b) Schachbrettmuster nach der Kalibrierung

Abbildung 2: Schachbrettmuster vor und nach der Kalibrierung

und nach der Kalibrierung zu erkennen. Das liegt daran, dass die Kamera relativ verzerrungsfrei aufnimmt. Wenn man die Bilder genauer betrachtet, fällt allerdings auf, dass die Abbildung 2a eine leichte Wölbung aufweist, welche in Abbildung 2b nicht mehr vorhanden ist.

Die Werte der Kalibrierung werden nun im Programmcode `rectified_image_node.cpp` gespeichert. Anschließend wird mit der OpenCV-Funktion `initUndistortRectifyMap()` die Matrizen für Verzerrung und Rektifizierung berechnet.

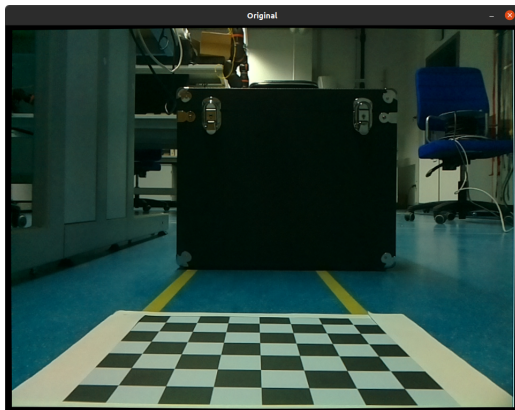
Um das Bild der Kamera zu erhalten wird ein ROS-Topic mit dem entsprechenden Bild abonniert. Jedes mal, wenn ein Bild von der Kamera veröffentlicht wird, wird eine Funktion aufgerufen. Diese erhält als Übergabeparameter das Bild als ROS-Nachricht. Dieses wird in eine OpenCV-Matrix umgewandelt um Matrixoperationen durchführen zu können. Mit der OpenCV-Funktion `remap()` wird dann das originale Bild mithilfe der Matrizen für Verzerrung und Rektifizierung in ein unverzerrtes Bild gewandelt. Dieses wird dann in ein ROS-Topic publiziert, damit es für andere Knoten zur Verfügung steht.

Damit sind die ersten beiden Aufgaben aus der Aufgabenstellung erfüllt.

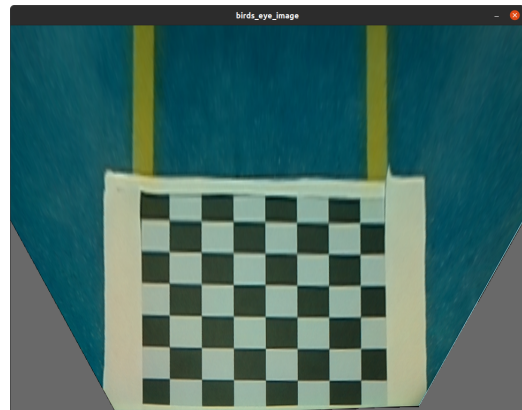
3.2.2 Bird's-Eye-View

Für das Bird's-Eye-View-Bild wird ein eigener Programmcode mit dem Namen `birdseye_node.cpp` geschrieben. In diesem wird das rektifizierte Bild abonniert. Anschließend wird mithilfe der Homographiematrix das Bild transformiert. Die Homographiematrix berechnet sich aus vier Bildpunkten die zu vier Objektpunkten verzerrt werden sollen. Die Bildpunkte lassen sich ermitteln, in-

dem ein Schachbrett mittig vor die Kamera auf den Boden gelegt wird. Mit der OpenCV-Funktion `findChessboardCorners()` werden dann die Pixelwerte der Eckpunkte des Schachbrettes bestimmt. Die Objektpunkte werden empirisch ermittelt. Sind die Punkte einmal bekannt, werden diese im Code gespeichert. Dann kann mit der Funktion `getPerspectiveTransform()` die Homographiematrix berechnet werden. Ist die Homographiematrix bekannt, lässt sich das Bird's-Eye-View über die Funktion `warpPerspective()` errechnen und anschließend auf einem ROS-Topic publizieren.



(a) Schachbrettmuster in der Originalperspektive



(b) Schachbrettmuster in der Vogelperspektive

Abbildung 3: Schachbrettmuster Original- und Vogelperspektive

Die Abbildung 3a zeigt das Schachbrett in der Originalperspektive des Roboters. Hierbei ist auch gut zu erkennen, dass die gelben Linien aufeinander zugehen. Durch die Umwandlung der Perspektive ergibt sich Abbildung 3b. Hier ist deutlich zu erkennen, dass die gelben Linien nun parallel verlaufen.

Wie beschrieben sind somit die Aufgaben drei und vier aus der Aufgabenstellung erfüllt.

3.2.3 Bird's-Eye-View mit LiDAR-Punkten

Für die Auswertung des LiDARs wird ein Programmcode mit dem Namen `birdseye_with_lidar_node.cpp` geschrieben. Dieser abonniert die ROS-Topics für den LiDAR und für das Bird's-Eye-View. Die Kameramatrix, die Homographiematrix und eine Transformationsmatrix werden ebenfalls im Code hinterlegt, um die LiDAR-Punkte in der gleichen Art zu transformieren, wie das Bird's-Eye-View. Die Transformationsmatrix wird benötigt, da das Koordinatensystem des LiDARs eine andere Orientierung und Position als die Kamera hat. Die Transformationsmatrix ist eine ho-

mogenen 4×4-Matrix, die aus einer Rotationsmatrix (3×3) und einem Translationsvektor besteht. Die allgemeine Form von T ist:

$$T = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (8)$$

Hierbei ist die 3x3-Matrix beginnend von oben links die Rotationsmatrix R und der Translationsvektor t ergibt sich aus dem letzten Element der jeweiligen Zeile. Beim Turtlebot ergibt sich folgende Matrix:

$$T = \begin{bmatrix} 0.0 & -1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & -1.0 & -0.056 \\ 1.0 & 0.0 & 0.0 & -0.075 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix} \quad (9)$$

Wobei die Rotationsmatrix R und der Translationsvektor t wie folgt aussehen:

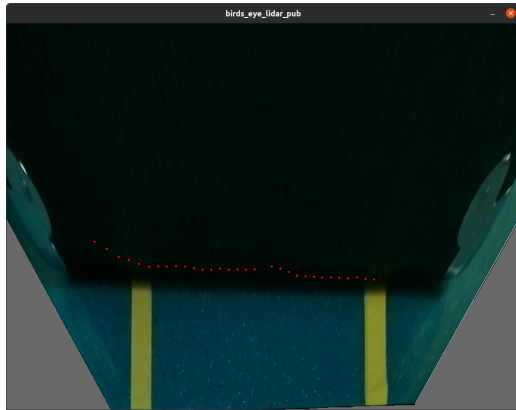
$$R = \begin{bmatrix} 0.0 & -1.0 & 0.0 \\ 0.0 & 0.0 & -1.0 \\ 1.0 & 0.0 & 0.0 \end{bmatrix} \quad t = \begin{bmatrix} 0.0 \\ -0.056 \\ -0.075 \end{bmatrix} \quad (10)$$

Die erste Zeile der Rotationsmatrix sagt aus, dass die y-Achse der Kamera entgegen der Richtung der x-Achse des LiDARs zeigt. Die zweite Zeile sagt aus, dass die z-Achse der Kamera entgegen der Richtung der y-Achse des LiDARs zeigt. Die dritte Zeile gibt an, dass die x-Achse der Kamera in Richtung der z-Achse des LiDARs zeigt. Dementsprechend ist das LiDAR um -90° um die x-Achse und um -90° um die z-Achse der Kamera gedreht.

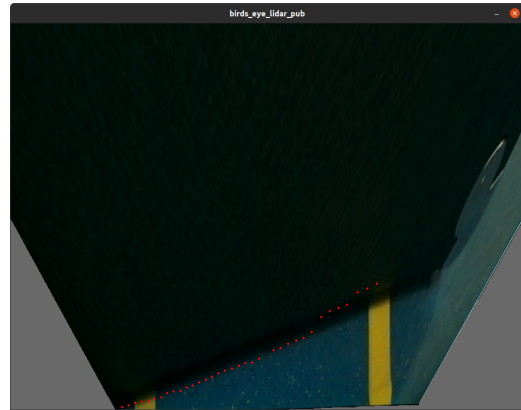
Die Elemente des Translationsvektors bedeuten, dass das Koordinatensystem vom LiDAR -0.056 in y-Richtung und -0.075 in z-Richtung vom Koordinatensystem der Kamera liegt, also 5,6cm höher und 7,5cm dahinter. (vgl. [14])

Die gemessenen Entfernungen des LiDARs werden dauerhaft abgefragt und daraus ergeben sich mehrere Ergebnisse. Zum einen werden die Punkte, die sich vor dem Roboter befinden mit der Transformationsmatrix in das Koordinatensystem der Kamera umgewandelt. Anschließend werden die Punkte mit der Kameramatrix verrechnet. Diese Punkte werden darauffolgend mit der Homographiematrix multipliziert, um die transformierten Punkte zu erhalten. Diese werden in das Bird's-Eye-View-Bild eingezeichnet. Zum anderen wird für die spätere Fahrt vom Roboter die minimale Distanz im Bereich von 90° vor dem Roboter und einmal rundherum gespeichert und veröffentlicht.

Außerdem wird links und rechts die minimale Distanz ermittelt, um die Richtungsangabe für die Hindernisumfahrung zu machen, was auch publiziert wird.



(a) Lidarpunkte in Bird's-Eye-View
(gerades Objekt)



(b) Lidarpunkte in Bird's-Eye-View
(schräges Objekt)

Abbildung 4: Lidarpunkte in Bird's-Eye-View

Die Abbildung 4a zeigt die eingezeichneten LiDAR-Punkte in der Bird's-Eye-View bei einem Objekt, welches gerade vor dem Roboter platziert wurde. Die Abbildung 4b zeigt die wiederum eingezeichneten LiDAR-Punkte in der Bird's-Eye-View bei einem Objekt, welches schräg vor dem Roboter platziert wurde. Hier ist gut zu erkennen, dass auch bei schrägem Objekt die Punkte trotz dessen richtig eingezeichnet werden.

Somit zeigt sich, dass die Aufgaben fünf und sechs aus der Aufgabenstellung erfüllt sind.

3.2.4 Fahrbahn folgen und Hindernis ausweichen

Für die letzte Pflichtaufgabe wird ein neuer Programmcode mit dem Namen `follow_line_node.cpp` erstellt. Dieser abonniert das Bird's-Eye-View für die Bilderkennung, das ROS-Topic `/odom` für die Odometry-Parameter, um Drehungen zu messen und die publizierten Werte der `birdseye_with_lidar_node.cpp`, um den Abstand nach vorne und rundherum zu erhalten.

Die Verfolgung der Fahrbahn und das Hindernisumfahren wird in mehrere Abschnitte unterteilt. Zuerst muss die Fahrbahn erkannt werden. Dies wurde mit einem HSV-Filter durchgeführt, da die Linien aus gelbem Kreppband bestehen. Dabei wurde das Bird's-Eye-View eingelesen und dann eine HSV-Maske drauf angewendet, der nach einem bestimmten Gelbwert filtert. Anschließend wird eine Morphologische-Operation durchgeführt. Hierbei wird Rauschen aus dem Bild entfernt und

die Genauigkeit der gelben Linien erhöht. Darauffolgend wird das Bild nach Konturen durchsucht. Dies geschieht mit der OpenCV-Funktion `findContours()`. Die erkannten Konturen werden anschließend nach der Größe gefiltert, damit zu kleine Konturen nicht berücksichtigt werden.

Nachdem die Konturen und damit die Fahrlinien erkannt wurden, muss als nächstes die Differenz zur Bildmitte berechnet werden. Sind zwei oder mehr Konturen vorhanden, werden vom jeweiligen Bildrand die am weitesten linke und rechte Kontur verwendet. Dann wird von jeder Kontur jeweils die horizontale Mitte bestimmt und anschließend die Mitte der beiden Mittellinien gebildet. Falls nur eine Kontur und damit nur eine Linie erkannt wurde, wird die horizontale Mitte dieser Kontur berechnet. Zusätzlich wird ein Faktor zu der Differenz multipliziert, damit der Roboter sich stärker dreht und somit schneller wieder beide Linien findet.

Zur Veranschaulichung der Bilderkennung wurde diese einmal an einer Geraden, an einer Kurve und an einer Kreuzung durchgeführt. Die Ergebnisse sind den Abbildungen 6, 7, 8 und 9 im Anhang dargestellt. Die Abbildungen aus Abbildung 6 zeigen die originalen Bird's-Eye-Views der Kamera. Hier ist zu erkennen, dass auch unterschiedliche Beleuchtungen und Farben von Krepp verwendet wurden. Nach dem Konvertieren in den HSV-Farbraum, ergeben sich die Abbildung 7a, Abbildung 7b und Abbildung 7c. Hierbei ist gut zu erkennen, dass vor allem die Linien deutlich hervorgehoben wurden. Durch das Anwenden der HSV-Maske, welche das Bild in ein binäres konvertiert und den Morphologischen-Operationen, die das Rauschen entfernen und kleine Lücken in den Objekten bzw. Linien füllen, ergeben sich Abbildung 8a, Abbildung 8b und Abbildung 8c. Hier sind nun ausschließlich die Linien zu sehen. Ein Hintergrundrauschen wurde vollständig herausgefiltert. Anhand des durch die Morphologische-Operation erstellten Bildes lassen sich nun die Konturen erkennen (siehe Abbildung 9). Diese werden durch ein rotes Rechteck dargestellt. Die horizontale Mitte der beiden äußeren Konturen wird durch die blauen Linien verdeutlicht. Die grüne Linie ist die sich ergebene Mitte der beiden blauen Linien. Abbildung 9a, Abbildung 9b und Abbildung 9c lässt sich entnehmen, dass das Erkennen der Konturen, das Berechnen der Mitten der Konturen und das Ermitteln der Mitte der äußeren Konturen gut funktioniert.

Zuletzt wird versucht, den Fehler mittels eines PD-Reglers auszugleichen und somit den Linien zu folgen. Damit wurde die Fahrbahnerkennung und das Fahren realisiert.

Für die Hinderniserkennung und das Umfahren wird auf die veröffentlichten Werte der `birdseye_with_lidar_node.cpp` zurückgegriffen. Dadurch wird erkannt, wenn ein Objekt einen definierten Abstand nach vorne unterschreitet. Ist dies der Fall, hält der Roboter zunächst an. Wird das Objekt innerhalb einer definierten Zeit wieder entfernt, kann der Turtlebot unbeirrt weiterfahren. Ist

dies allerdings nicht der Fall, muss der Turtlebot dieses Objekt umfahren. Dazu dreht er sich um 90° in die Richtung, in der mehr Platz ist. Diese Information wurde ebenfalls über eine ROS-Topic abonniert. Nach der Drehung erfolgt das Umfahren des Hindernisses. Ist die Fahrbahn wieder zu erkennen, wird zu dieser gefahren und das Linienverfolgen wird fortgesetzt.

Um das Verhalten des Roboters übersichtlich darzustellen, wurde eine State-Machine erstellt. Hierbei gibt es die Zustände:

- *SEARCH*: In diesem Zustand sucht der Roboter nach einer Linie, in dem er sich langsam um die eigene Achse dreht. Wenn eine Linie erkannt wird, wird in den Zustand *FOLLOW_LINE* gewechselt. Wird während einer ganzen Drehung (360°) keine Linie erkannt, hält der Roboter an und wartet, bis er vor eine Linie gestellt wird.
- *FOLLOW_LINE*: In diesem Zustand wird das eigentliche Folgen der Linien vollzogen. Solange kein Hindernis vor dem Roboter ist, wird mit konstanter Geschwindigkeit gefahren und die Winkelgeschwindigkeit entsprechend des Linienverlaufs angepasst. Sind keine Linien mehr in dem Kamerabild wechselt der Roboter in den Zustand *SEARCH*. Falls ein Hindernis vor dem Roboter auftaucht und der Abstand geringer als 40cm ist, wechselt er in den Zustand *WAIT_CHECK*.
- *WAIT_CHECK*: In diesem Zustand wird geprüft, ob das Hindernis nach Ablauf einer bestimmten Zeit immer noch vor dem Roboter ist. Dabei hält der Roboter an und startet einen Timer. Wenn während des Ablaufens das Hindernis entfernt wird, wechselt der Roboter zurück zu *FOLLOW_LINE*. Wenn der Timer abgelaufen ist, wird in den Zustand *TURN_90* gewechselt.
- *TURN_90*: In diesem Zustand dreht sich der Roboter um 90° , abhängig davon, ob mehr Platz nach links oder rechts vorhanden ist, und wendet sich in die entsprechende Richtung. Anschließend wird in den Zustand *AVOID_OBSTACLE* gewechselt.
- *AVOID_OBSTACLE*: Dieser Zustand dient dem eigentlichen Umfahren von Hindernissen. Hierbei wird ein Abstand zum Hindernis gehalten. Falls nun etwas vor dem Roboter auftaucht, hält er an und wartet, bis es entfernt wird. Der Turtlebot umfährt das Hindernis solange, bis die Linien wieder im Bird's-Eye-View liegen. Dann wird in den Zustand *FOLLOW_LINE* gewechselt.

Das Verhalten der Zustände lässt sich anhand von Abbildung 10 im Anhang gut erkennen. Somit ist zu jedem Zeitpunkt genau bestimmbar, in welchem Zustand der Roboter sich befindet und welche Schritte durchgeführt werden.

Um das Zusammenspiel der Programmcodes und damit Nodes zu verdeutlichen, wurde ein Programmablaufdiagramm erstellt. Dieses stellt die Beziehung der Nodes und ROS-Topics dar. Das Diagramm ist der Abbildung 11 im Anhang zu entnehmen. Die roten Pfeile symbolisieren Publisher und die blauen stehen für Subscriber.

Mit diesem Programm kann nun der Roboter zum einen einer Spur aus zwei Linien folgen, was die letzte Aufgabe der Pflichtaufgaben aus der Aufgabenstellung ist. Zudem kann der Roboter auch Hindernisse erkennen und ihnen ausweichen, wodurch ebenfalls die erste Zusatzaufgabe der Aufgabenstellung erfüllt wurde.

Damit die einzelnen Nodes einfacher gestartet werden können, wurde eine Datei geschrieben, die mit dem Befehl `roslaunch` (siehe Unterabschnitt 2.2) aufgerufen werden kann. Der Inhalt der Datei ist in Codeauszug 1 dargestellt. Statt jeden Node einzeln zu starten, muss dadurch nur noch die `launch`-Datei aufgerufen werden.

4 Ergebnisse

Nach der Umsetzung wird im Folgenden der Verlauf des Projekts zusammengefasst, die gewonnenen Erkenntnisse reflektiert und abschließend ein Ausblick gegeben.

4.1 Fazit

Wie bereits in der Umsetzung beschrieben, wurde aus der originalen Aufnahme zunächst das Bild rektifiziert, um anschließend ein Bird's-Eye-View zu generieren. Mit diesem Bild und dem LiDAR-Scanner wurden die durch Hindernisse verursachten LiDAR-Punkte in das Bird's-Eye-View eingezeichnet. Darüber hinaus wurde ein Algorithmus entwickelt, der die Fahrbahn erkennt, um dieser zu folgen. Sobald ein Hindernis erkannt wird, erfolgt eine Umfahrung, bevor die Fahrspurverfolgung fortgesetzt wird.

Somit wurden alle Pflichtaufgaben sowie die erste Zusatzaufgabe erfolgreich umgesetzt.

Allerdings funktioniert der Turtlebot derzeit nur unter bestimmten Bedingungen. Zum einen gibt es Einschränkungen durch den Roboter selbst: So kann dieser nur auf ebenen Oberflächen fahren, und der LiDAR befindet sich auf einer vordefinierten Höhe, wodurch nur bestimmte Hindernisse erkannt werden können. Zum anderen muss das Sonnenlicht momentan noch reduziert werden, um Störungen durch Spiegelungen in der Bildverarbeitung zu vermeiden.

4.2 Learnings

Im Rahmen des Projekts wurde vertieftes Wissen zu ROS und OpenCV erarbeitet, insbesondere im Hinblick auf deren Aufbau, Anwendung und Programmierung. Es wurde ein tiefgehendes Verständnis für die vielfältigen Möglichkeiten entwickelt, die ROS bietet, sowie für dessen Rolle im Kontext der Robotik. Zudem wurde ein grundlegendes Verständnis dafür geschaffen, wie moderne Fahrzeuge einer Fahrbahn folgen und welche Herausforderungen bei der Implementierung solcher Systeme auftreten können. Besonders im Bereich der digitalen Bildverarbeitung wurden wichtige Erkenntnisse gewonnen. So funktionierte die Bestimmung des Schwellenwerts mittels der OTSU-Methode zur Binarisierung des Bildes einwandfrei, solange Linien vorhanden waren. Fehlen jedoch Vordergrundobjekte, wird der Schwellenwert unbrauchbar. Daher wurde die Binarisierung schließlich über eine HSV-Maske realisiert, um zwischen den Zuständen *SEARCH* und *FOLLOW_LINE* effizient wechseln zu können.

4.3 Ausblick

Das Projekt bietet zahlreiche Möglichkeiten zur Erweiterung. Zunächst könnten die verbleibenden beiden Zusatzaufgaben umgesetzt werden. Zudem könnten die Regelparameter weiter optimiert und die Fehleranfälligkeit weiter minimiert werden. Nach erfolgreichem Sticking der Karte aus der letzten Zusatzaufgabe könnte die Fahrt ohne Kamera fortgesetzt werden, basierend allein auf den aufgenommenen Linien in der Karte, was die Fehleranfälligkeit deutlich verringern könnte. Darüber hinaus könnte eine erweiterte Objekt- und Hinderniserkennung, -unterscheidung und -vermeidung hinzugefügt werden, entweder allein über die Kamera oder durch Kombination mit dem LiDAR.

Literatur

- [1] HanCheol Cho YoonSeok Pyo, RyuWoon Jung und TaeHoon Lim. *ROS Robot Programming*. An introduction to Robot Operating System (ROS) with practical examples and tutorials. ROBOTIS Co. Ltd., 2017. ISBN: 979-11-962307-1-5. URL: <https://www.robotis.com> (besucht am 19. 11. 2024).
- [2] ROBOTIS. *TurtleBot3 eManual*. Offizielle Dokumentation des TurtleBot3 von ROBOTIS. URL: <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/> (besucht am 22. 02. 2025).
- [3] Gary Bradski und Adrian Kaehler. *Learning OpenCV*. A working guide to the Open Source Computer Vision Library (OpenCV), providing a general background to the field of computer vision. O'Reilly Media, Inc., 2008. ISBN: 978-0596516130. URL: <http://www.oreilly.com/catalog/9780596516130> (besucht am 19. 11. 2024).
- [4] Porsche Engineering. „Porsche nutzt ROS für autonome Fahrfunktionen“. In: *Porsche Engineering Magazin* (2021). Zugriff am 24. Februar 2025. URL: <https://newsroom.porsche.com/de/2024/innovation/porsche-engineering-roboter-betriebssystem-ros-jupiter-versuchsfahrzeug-35898.html>.
- [5] ADAC. *Spurhalteassistenten im Vergleich: Funktionen, Vorteile und Unterschiede*. Zugriff am 24. Februar 2025. 2024. URL: <https://www.adac.de/rund-ums-fahrzeug/ausstattung-technik-zubehoer/assistentensysteme/spurhalteassistenten-vergleich/>.
- [6] Bosch Mobility. *Spurassistentensysteme*. Zugriff am 24. Februar 2025. 2025. URL: <https://www.bosch-mobility.com/de/loesungen/assistentensysteme/spurassistentensysteme/>.
- [7] Robert Bosch GmbH. *Automotive Handbook*. 10. Aufl. Springer Vieweg, 2023. ISBN: 978-0-7680-9567-8.
- [8] ROS Wiki. *ROS Concepts*. URL: <https://wiki.ros.org/ROS/Concepts> (besucht am 23. 02. 2025).
- [9] OpenCV. *homography matrix*. URL: https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html (besucht am 24. 02. 2025).
- [10] MathWorks. *Graythresh*. URL: <https://de.mathworks.com/help/images/ref/graythresh.html> (besucht am 24. 02. 2025).

- [11] OpenCV. *Thresholding - inRange*. URL: https://docs.opencv.org/3.4/da/d97/tutorial_threshold_inRange.html (besucht am 24.02.2025).
- [12] OpenCV. *Morphological Operations*. URL: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html (besucht am 24.02.2025).
- [13] Thomas Linkugel. *Projektarbeiten*. 2024.
- [14] OpenCV. *Camera Calibration and 3D Reconstruction*. URL: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html (besucht am 24.02.2025).
- [15] Murat Çalis. *Roboter mit ROS: Bots konstruieren und mit Open Source programmieren*. Eine umfassende Einführung in die Konstruktion und Programmierung von Robotern mit dem Robot Operating System (ROS). dpunkt.verlag, 2020. ISBN: 978-3-86490-567-4. URL: <https://www.edv-buchversand.de/product/dp-567/Roboter%20mit%20ROS>.

5 Anhang

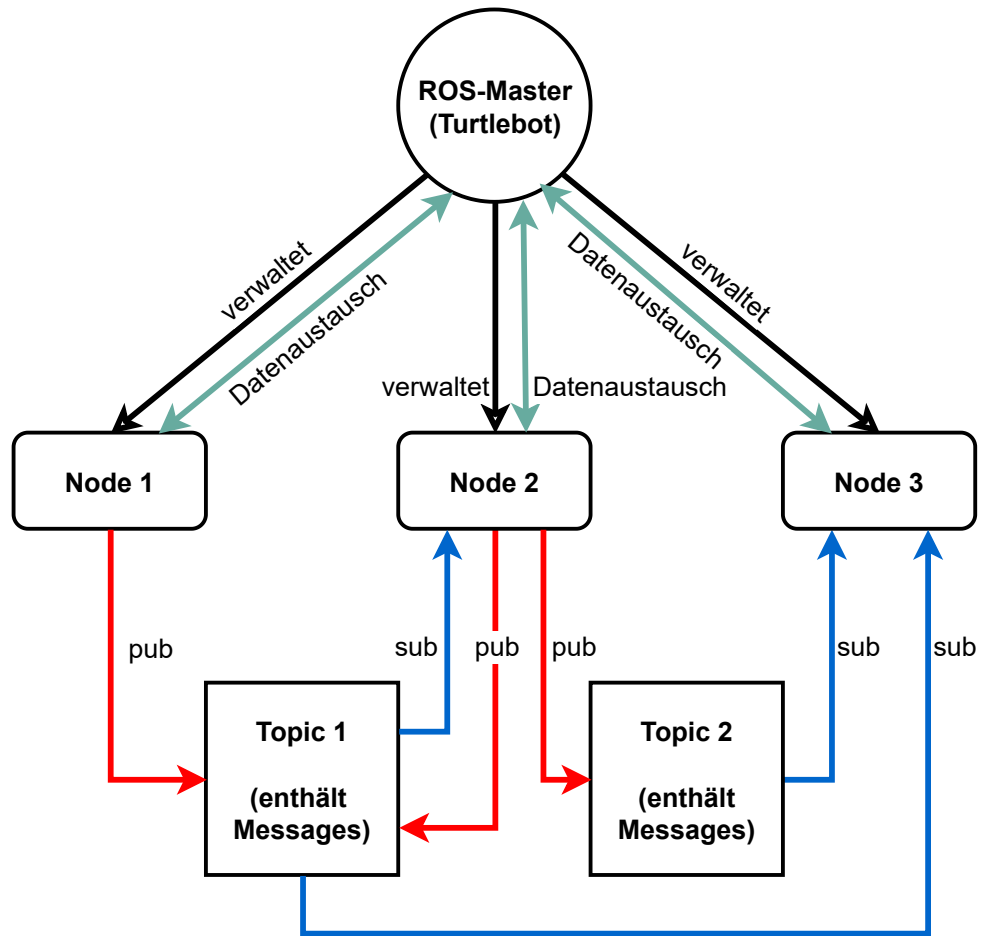
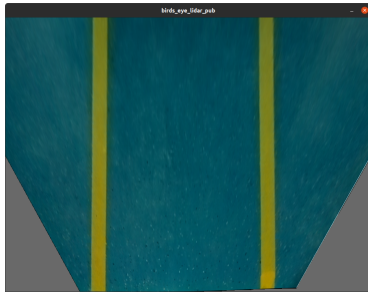
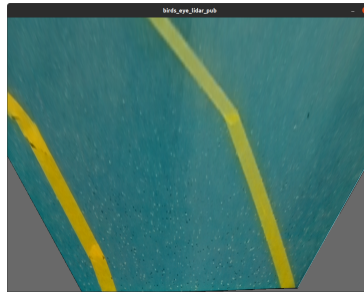


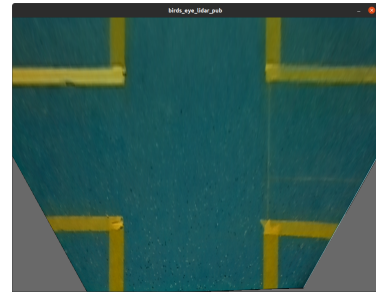
Abbildung 5: ROS-Kommunikationsdiagramm [15, vgl. S. 18-21]



(a) Gerade im
Bird's-Eye-View

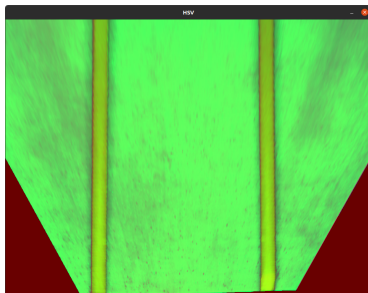


(b) Kurve im
Bird's-Eye-View

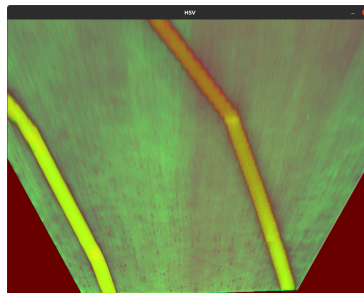


(c) Kreuzung im
Bird's-Eye-View

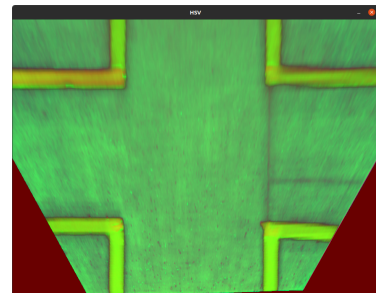
Abbildung 6: Verschiedene Streckenabschnitte im Original-Bird's-Eye-View



(a) Gerade mit
HSV-Maske



(b) Kurve mit
HSV-Maske



(c) Kreuzung mit
HSV-Maske

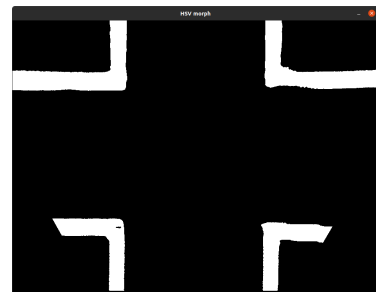
Abbildung 7: Verschiedene Streckenabschnitte im HSV-Farbraum



(a) Gerade nach
Morphologische-Operation

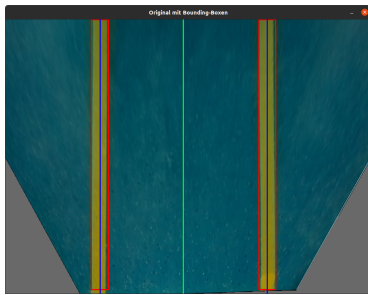


(b) Kurve nach
Morphologische-Operation

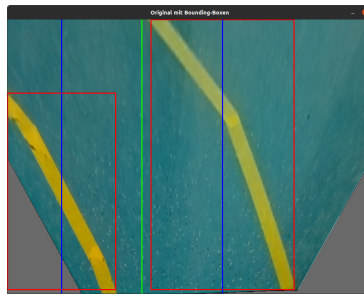


(c) Kreuzung nach
Morphologische-Operation

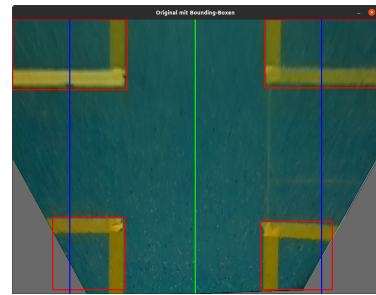
Abbildung 8: Verschiedene Streckenabschnitte nach Morphologische-Operation



(a) Gerade mit Konturen und berechneter Mitte



(b) Kurve mit Konturen und berechneter Mitte



(c) Kreuzung mit Konturen und berechneter Mitte

Abbildung 9: Verschiedene Streckenabschnitte mit Konturen und berechneter Mitte

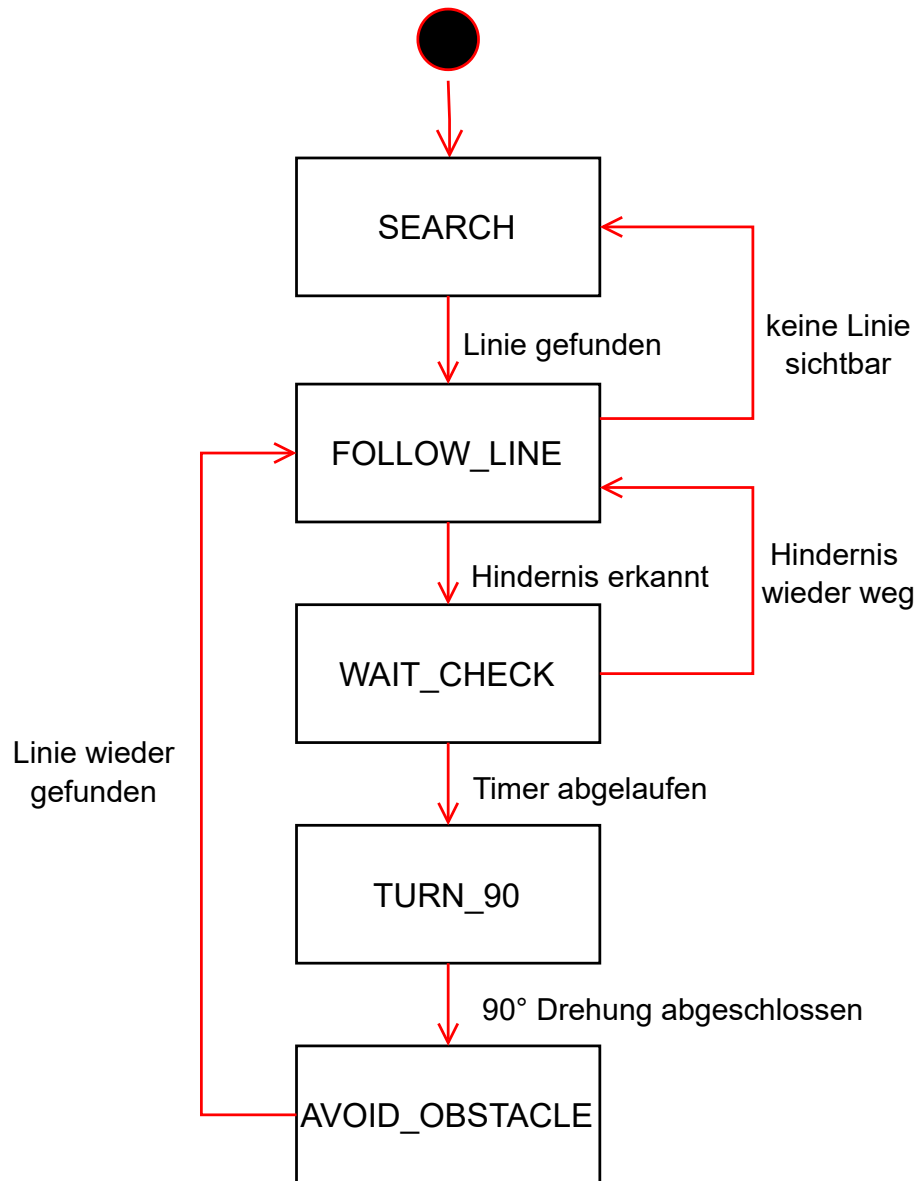


Abbildung 10: Zustandsdiagramm

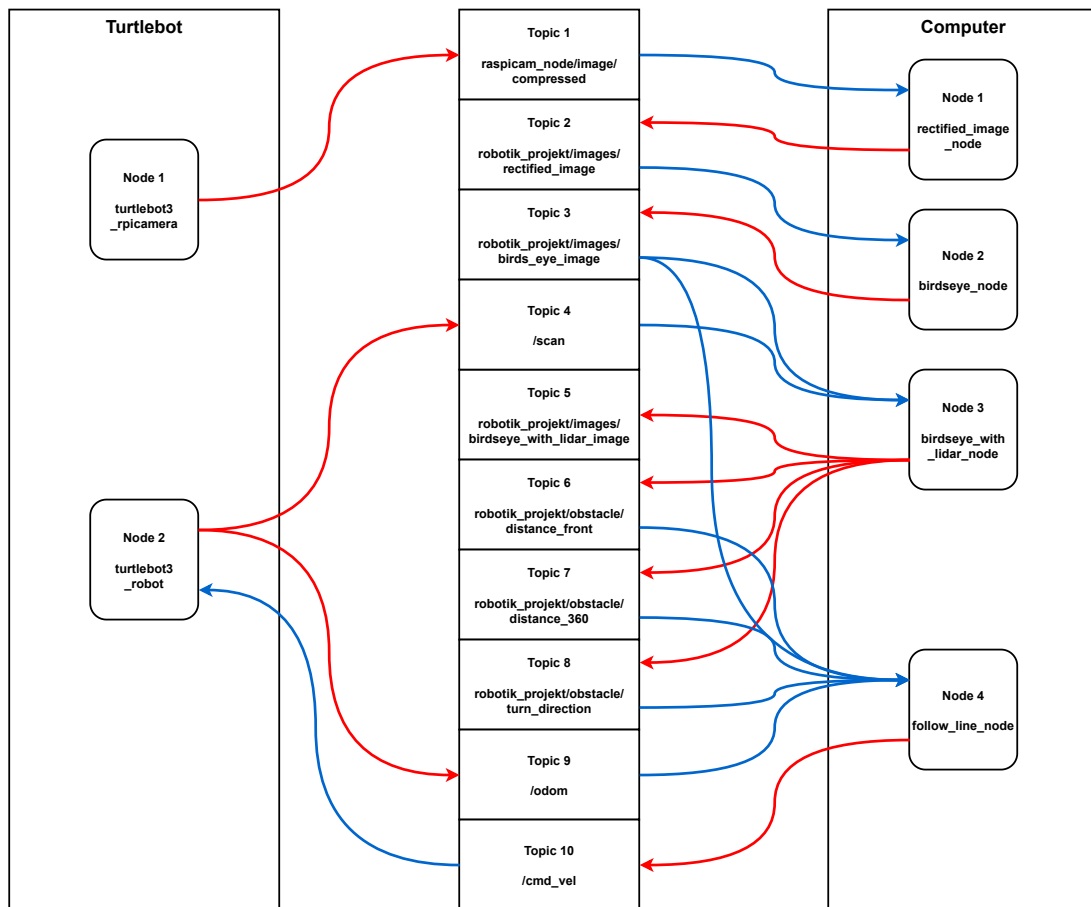


Abbildung 11: Programmablaufdiagramm

```
1 <launch>
2   <!-- Startet den rectified_image_node -->
3   <node pkg="robotik_projekt_ws24_nvs_js" type="rectified_image_node" name=
      "rectified_image_node" output="screen" />
4
5   <!-- Startet den birdseye_node -->
6   <node pkg="robotik_projekt_ws24_nvs_js" type="birdseye_node" name="
      birdseye_node" output="screen" />
7
8   <!-- Startet den birdseye_with_lidar_node -->
9   <node pkg="robotik_projekt_ws24_nvs_js" type="birdseye_with_lidar_node"
      name="birdseye_with_lidar_node" output="screen" />
10
11  <!-- Startet den follow_line_node -->
12  <node pkg="robotik_projekt_ws24_nvs_js" type="follow_line_node" name="
      follow_line_node" output="screen" />
13 </launch>
```

Codeauszug 1: Launch-Datei